PulseFit Gym API

Project guide: how it works, how to run it from a fresh *git clone*, how to test it in Postman one request at a time, and a script for the video demonstration

Python Flask · MongoDB · JWT · Postman
COM661 Full Stack Strategies and Development, CW1
Repository: github.com/1-Ibrahim-CS/GymPOST_Project
17 October 2026

Contents

1. What the project is
2. Project structure
3. How a request flows through the code
4. Data design and key algorithms
5. Running the project from a fresh git clone
6. Testing with Postman, one request at a time
7. Video demonstration script

Appendix A. All 153 Postman requests in run order

1. What the project is

PulseFit is the back end for a gym chain with several branches. It is a RESTful API written in **Python Flask** and stored in **MongoDB**. It has no user interface: every feature is used by sending HTTP requests, which is what Postman is for. There are **51 endpoints** in **9 Blueprints**, all under `/api/v1.0`, tested by a Postman collection of **153 requests and 325 assertions**.

There are three kinds of user, and each one has a login account with a role:

Role	What they can do
Member	Browse, search and review 300 exercises; find the nearest branch; book and cancel class places (a class can never be overbooked); log workouts and see progress statistics.
Trainer	Create and edit exercises; create and manage the classes they teach; record attendance.
Admin	Everything: branches, members and memberships, any class, and the management reports.

The data comes from two Kaggle datasets (gym exercises and gym members). Branches, trainers, classes, bookings, workout history and reviews are generated on top of that by `Raw_Dataset/seed.py`. The seeded database is deliberately kept small: **6 branches, 300 exercises, 100 members, 240 classes and 119 user accounts** (1 admin, 18 trainers, 100 members).

2. Project structure

```
app.py                creates the Flask app, registers the 9 Blueprints, JSON error handlers
globals.py            reads .env, opens the MongoDB connection, loads OPTIONS and SCHEMAS from it
decorators.py         @jwt_required and @role_required(...) - the access checks
requirements.txt      Python packages (Flask, pymongo, PyJWT, bcrypt, flask-cors, python-dotenv)
.env.example          template for .env (MONGODB_URI, SECRET_KEY) - .env itself is not in git
utils/
  validation.py       schema-based checking of JSON bodies and query strings, ?pn=&ps= paging
  serialize.py        JSON provider: ObjectId -> string, datetime -> ISO 8601
blueprints/           one folder per resource, each with <name>.py (members.py also has BMI + new member)
  auth  branches  exercises  reviews  members  workouts  classes  bookings  reports
Raw_Dataset/
  seed.py             builds gymDB from the CSVs, plus the options and schemas collections
  megaGymDataset.csv, gym_members_exercise_tracking.csv
postman/
  GymAPI.postman_collection.json     the 153 test requests
  GymAPI.postman_environment.json    the "PulseFit Local" environment (baseUrl, passwords)
export/               mongoexport JSON of every collection (for the submission)
ENDPOINTS.md          table of all 51 endpoints
DESIGN_NOTES.md       design decisions, algorithms and alternatives
```

3. How a request flows through the code

```
Postman --> app.py --> Blueprint route --> decorators.py --> utils/validation.py --> MongoDB
                                          (token + role)     (check the input)      (globals.py)
```

- **app.py** creates the app and registers one Blueprint per resource. Its error handlers make every error, even 404 and 405, come back as JSON: `{"error": "...", "details": {...}}`.
- **globals.py** loads the **.env** file and opens a single MongoClient shared by every request. The collections are `users`, `members`, `branches`, `exercises`, `classes`, `blacklist`, `options` and `schemas`. At start-up it also reads two documents: `OPTIONS`, the allowed values (plans, genders, workout types, body parts, equipment ...), and `SCHEMAS`, the validation rules for every request body, e.g.

`SCHEMAS["branch_create"]`. Neither is hard-coded in the blueprints; seed.py puts both in the database.

- **decorators.py**: `@jwt_required` reads the `x-access-token` header, checks the JWT signature and expiry, and rejects tokens that have been logged out (they are in `blacklist`). `@role_required("admin", ...)` also checks the role and returns 403 for the wrong role.
- **utils/validation.py**: `validate_body()` and `validate_query()` check the input against a schema (from SCHEMAS for bodies) and return `(data, errors)`. If there are errors, the route returns 400 with the errors as details. Unknown fields are rejected.
- **The route** builds a MongoDB query, runs it and returns `make_response(jsonify(...), status)`.

Worked example: GET /members (blueprints/members/members.py)

1. `@role_required("admin")`: only an admin token gets in. A member token gets 403, no token gets 401.
2. `validate_query({...})` checks `?branch=&status=&plan=&workout_type=&gender=&q=&sort=&pn=&ps=`. A wrong value, e.g. an invalid plan, returns 400.
3. Each filter is added only if the client sent it: `if "branch" in args: match["home_branch_id"] = args["branch"]`. `status` is worked out from the membership end date, so it is always correct for today. `q` is a case-insensitive regex on name or email.
4. One aggregation pipeline: `$match` (filters) → `$addFields` (workout_count) → `$project` (hide the workouts array) → `$sort` → `$facet` (one page of data plus the total count in the same query).
5. The response is `{"data": [...], "pagination": {"page", "page_size", "total", "pages"}}`.

Almost every route in the project follows this same pattern: **check access** → **validate** → **build the query** → **run it** → **return JSON**.

4. Data design and key algorithms

The main design decision in MongoDB is what to **embed** inside a document and what to **reference** by id. The rule used: embed data that is always read with its parent, belongs to it and has a limited size; reference data that has its own life or would grow without limit.

Collection	Embedded	Referenced	Why
branches	address, opening_hours, facilities[], location (GeoJSON Point)	–	Small fixed sub-objects. The GeoJSON point is needed for the 2dsphere index and the nearest-branch search.
exercises	reviews[]	reviews.member_id → members	A review only makes sense with its exercise; the average stars need no join. member_name is copied in.
members	body_metrics, membership, workouts[] (each with exercises[])	home_branch_id → branches; workouts.exercises.exercise_id → exercises	Workout history is private and read as a whole, so one aggregation produces all the statistics.
classes	bookings[]	branch_id, trainer_id, bookings.member_id	Bookings are limited by capacity (≤ 100). Keeping them in the class document makes atomic booking possible.
users	specialities[] (trainers)	member_id → members; branch_id → branches	Login data (bcrypt hash, role) is kept apart from profiles; staff have no member record.

Collection	Embedded	Referenced	Why
blacklist	–	–	One document per logged-out token, deleted automatically by a TTL index once it expires.
options	plan_days{} and the allowed-value lists	–	One lookup document with every allowed value the API validates against, so the lists live in the database, not in the code.
schemas	one validation schema per request body	–	The rules for every JSON body (register, branch_create, class_update, workout ...), built by seed.py, so the blueprints hold no schema definitions.

Key algorithms worth explaining in the video

- **Atomic class booking** (blueprints/bookings/bookings.py). A single `update_one` whose filter requires the class to be scheduled and in the future, `booked_count < capacity`, and no existing booking by this member; the update pushes the booking and increments `booked_count`. A single-document update is atomic in MongoDB, so two members can never take the last place at the same time. The member must also have an active membership covering the class date.
- **Trainer clash check** (blueprints/classes/classes.py). A new or moved class returns 409 if the trainer already has a class that overlaps in time.
- **Geospatial search**: `GET /branches/near?lat=&lng=&km=` uses `$geoNear` on the 2dsphere index and returns `distance_km`, nearest first.
- **Search and filtering**: comma-separated filters become `$in`; `q` on exercises uses the full-text index on title and description.
- **Aggregation pipelines**: member stats (weekly breakdown, favourite type, most-used exercises) and the five admin reports (`popular-classes`, `branch-usage`, `peak-times`, `trainer-performance`, `membership-summary`).
- **Authentication**: login with HTTP Basic auth; password checked with bcrypt; a JWT valid for 8 minutes holds `user_id, username, role, member_id`; logout puts the token in the blacklist.

DESIGN_NOTES.md covers each of these in more depth, including the alternatives that were considered.

5. Running the project from a fresh git clone

A clone contains all the code, the two CSV datasets and the Postman files. It does **not** contain the `.env` file (it is in .gitignore because it holds the database password) or the `.venv` folder. So after cloning you create a virtual environment, install the packages, create .env, seed the database and start the server.

5.1 What you need installed

Tool	Version	Why
Git	any	To clone the repository.
Python	3.10 or later	Runs the API and seed.py. Tick “Add python.exe to PATH” when installing on Windows.
MongoDB	5.0 or later	Either MongoDB Community Server installed locally, or a free MongoDB Atlas cluster. 5.0+ is needed for $dateTrunc, $dateDiff and $lookup with a pipeline.
Postman	desktop app	To send the requests and run the test collection.
Node.js	optional	Only to run the tests from the command line with Newman.
MongoDB Database Tools	optional	Only for mongoexport (the export/*.json submission files).

5.2 Step by step (Windows PowerShell)

1. Clone the repository and go into it:

```
git clone https://github.com/1-Ibrahim-CS/GymPOST_Project.git
cd GymPOST_Project
```

2. Create and activate a virtual environment, then install the packages:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If PowerShell says *running scripts is disabled on this system*, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then activate again. In Command Prompt use `.venv\Scripts\activate.bat` instead. When it is active, the prompt starts with `(.venv)`.

3. Create the .env file from the template:

```
Copy-Item .env.example .env
```

4. Open .env and set the two values (see 5.3 for which MONGODB_URI to use):

```
MONGODB_URI=mongodb://127.0.0.1:27017
SECRET_KEY=put-any-long-random-string-here
```

5. Seed the database. This must be run from inside Raw_Dataset, because the script opens the CSV files by relative path:

```
cd Raw_Dataset
python seed.py
cd ..
```

It prints *Building data...*, the number of documents in each collection, *indexes created* and finally *Done. Every seeded account's password is: Password123!*. It **drops and rebuilds** the `gymDB` database every time, so only run it when you want

fresh data.

6. Start the API and leave this terminal open:

```
python app.py
```

You should see `Running on http://127.0.0.1:5001`. Press Ctrl+C to stop it.

macOS / Linux

```
git clone https://github.com/1-Ibrahim-CS/GymPOST_Project.git
cd GymPOST_Project
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # then edit MONGODB_URI and SECRET_KEY
cd Raw_Dataset && python seed.py && cd ..
python app.py
```

5.3 Choosing the database

Option	MONGODB_URI in .env	Notes
Local MongoDB	`mongodb://127.0.0.1:27017`	Install MongoDB Community Server and make sure the MongoDB service is running. Simplest for a demo, and no internet is needed.
MongoDB Atlas	`mongodb+srv://<user>:<password>@<cluster>.mongodb.net/`	In Atlas: create a free cluster, create a database user under *Database Access*, and allow your IP under *Network Access* (or 0.0.0.0/0 for testing). Copy the connection string from *Connect → Drivers*.
Atlas, already seeded	your existing Atlas URI	If the database is already seeded and the data is less than about 3 weeks old (see the note below), you can skip the seed step.

The database name is always `gymDB`. Both seed.py and app.py read the same .env file, so they always use the same database. A variable already set in the shell overrides the file.

> **Important.** seed.py builds the data around the day you run it: 8 weeks of past workouts, 4 weeks of past classes and **4 weeks of classes from the current week on**. Memberships are also dated from that day. If the data was seeded more than about 3 weeks ago, there are no upcoming classes left to book and some Postman tests will fail. **Re-run seed.py shortly before you record the video.**

5.4 Check that it works

Open these in a browser while app.py is running:

- `http://127.0.0.1:5001/` shows the heading *PulseFit Gym API*.
- `http://127.0.0.1:5001/api/v1.0/branches?ps=2` shows JSON with two branches and a pagination object.
- `http://127.0.0.1:5001/api/v1.0/exercises/filters` shows the body-part, equipment, level and type values.

5.5 Seeded logins

Every seeded account uses the password **Password123!**

Role	Username	Notes
admin	`admin`	Full access, including the reports.
trainer	`sophie.boyle`	18 trainers in total, 3 per branch. Usernames are the part of the email before @.
member	`oliver.robinson`	100 members. Usernames are firstname.lastname.

5.6 Troubleshooting

Problem	Cause and fix
`The options or schemas collection is empty - run Raw_Dataset/seed.py first`	The database has not been seeded (or was seeded by an older seed.py without these collections). Run seed.py, then start app.py again.
`ModuleNotFoundError: No module named 'flask'`	The virtual environment is not active, or the packages were not installed. Activate .venv and run pip install -r requirements.txt.
`ServerSelectionTimeoutError`	Python cannot reach MongoDB. Local: start the MongoDB service. Atlas: check the URI, the user's password, and that your IP is allowed under Network Access.
`FileNotFoundError: ...megaGymDataset.csv`	seed.py was not run from inside the Raw_Dataset folder.
Port 5001 already in use	Another copy of app.py is still running; close it. To use a different port, change port=5001 in the last line of app.py and baseUrl in the Postman environment.
Postman: *Could not send request / ECONNREFUSED*	app.py is not running, or the **PulseFit Local** environment is not selected, so {{baseUrl}} is empty.
`401 Token has expired`	Tokens last 8 minutes. Re-send the Login requests in folder 1.
A request shows {{memberToken}} or {{classId}} unresolved	An earlier request that saves that variable has not been sent yet. See section 6.2.
“List upcoming classes” or the booking tests fail	The seeded data is more than about 3 weeks old. Re-run seed.py.
`mongoexport: command not found`	Install MongoDB Database Tools and add its bin folder to PATH.

6. Testing with Postman, one request at a time

6.1 Import

1. Start the API (python app.py).
2. In Postman click **Import** and choose both files from the postman folder: `GymAPI.postman_collection.json` and `GymAPI.postman_environment.json`.
3. In the environment dropdown at the top right, select **PulseFit Local**. It sets `baseUrl = http://127.0.0.1:5001/api/v1.0` and the seed password.
4. Open the collection **PulseFit Gym and Fitness Class Booking API**. It has 10 folders, numbered in run order.

6.2 Why the order matters

Requests pass data to later requests through **collection variables**. Each request's **Post-response** script saves what it gets back, for example `pm.collectionVariables.set("adminToken", json.token)`. Later requests use it as `{{adminToken}}` in the `x-access-token` header or in the URL. The main ones:

Saved by	Variable(s)	Used for
1. Auth → Login as admin	adminToken, runId	Admin requests; runId makes test usernames unique
1. Auth → Login as trainer	trainerToken, trainerId	Creating exercises and classes
1. Auth → Current user (trainer)	branchId	The trainer's branch, used in many later requests
1. Auth → Register new member / Login as new member	memberId, memberToken	The main test member
1. Auth → Register / Login second member	otherMemberId, otherMemberToken	Testing “not yours” (403) cases
3. Exercises → Create exercise	exerciseId	Reviews, workouts, clean-up
5. Members → List members (admin)	seededMemberId	Stats and workouts of a member with history
6. Workouts → Log workout	workoutId	Get, update, delete workout
7. Classes → Create class (capacity 1)	classId	All the booking tests in folder 8
8. Bookings → Book class	bookingId	Get, cancel and attendance tests

Rule: always go top to bottom inside a folder, and folder 1 → 10. You can stop at any point, but you can't skip ahead past a request that saves something you need.

6.3 How to send and show one request

1. Click the request in the sidebar. Show the **method and URL**, and the **Headers** tab (`x-access-token: {{memberToken}}`) or **Authorization** tab (Basic Auth for login).
2. If it has a body, show the **Body** tab (raw JSON).
3. Click **Send**. Point out the **status code** (201 Created, 409 Conflict, ...) and the JSON response.
4. Open the **Test Results** tab in the response panel. Every test should be green (PASS).
5. Optionally open **Scripts** → **Post-response** to show the tests and the line that saves the token or id.
6. To see the current value of any variable, click the collection → **Variables** tab.

6.4 Things that will trip you up

- **Always start with folder 1, requests 1–11.** They log in the admin and trainer, register and log in two fresh test members, and set branchId. Without them, every later request fails.
- **Tokens expire after 8 minutes.** On *401 Token has expired*, re-send the Login requests in folder 1. (Re-sending Login as admin creates a new runId, so also re-send Register new member, Login as new member, Register second member and Login as second member.)
- **Folder 7 must come before folder 8.** Create class makes a class with only 1 place. That is why Book class expects `spaces_left: 0` and Book class - full expects 409.
- **Requests that expect an error are meant to fail.** A request named "... - forbidden" is supposed to return 403; its tests pass when it does.
- **Folder 10 cleans up.** It deletes the test exercise and test members and logs out. After it, start again from folder 1. The run deletes everything it creates, so it can be repeated without re-seeding.

6.5 Running the whole collection at once

In Postman: click the collection → **Run** → keep all 10 folders ticked, in order → **Run**. Expected: **153 requests, 325 tests passed, 0 failed**. Export the results for the submission from the runner's results screen.

From the terminal (with app.py running in another terminal):

```
# Newman (needs Node.js; npx downloads it on first use)
npx newman run postman/GymAPI.postman_collection.json `
    -e postman/GymAPI.postman_environment.json

# or the Postman CLI
postman collection run postman/GymAPI.postman_collection.json `
    -e postman/GymAPI.postman_environment.json

# optional HTML report like TestResults.html
npm install -g newman newman-reporter-htmlextra
newman run postman/GymAPI.postman_collection.json `
    -e postman/GymAPI.postman_environment.json `
    -r cli,htmlextra --reporter-htmlextra-export TestResults.html
```

The backtick ` at the end of a line continues the command in PowerShell. In bash or macOS use \ instead, or put each command on one line.

7. Video demonstration script

You don't need to show all 153 requests. For each area, show the main success case and one or two error cases, then end with the full collection run. Request names below are exactly as they appear in Postman.

#	Scene	What to send in Postman	What to point out
0	Intro and code tour	VS Code: app.py (Blueprints, error handlers), decorators.py, one route such as GET /members. Run python app.py.	Structure, request flow (section 3), the .env and seed.py.
1	Auth	**1. Auth**: Login as admin · Login - wrong password (401) · Register new member (201) · Login as new member · Current user (member) · Current user - missing token (401)	Basic Auth login, JWT in x-access-token, bcrypt passwords, 8-minute expiry.
2	Branches	**2. Branches**: List branches · List branches - filter by city and facility · Branches near a location · Create branch - member forbidden (403) · Create branch (201) · Delete branch - still in use (409)	Pagination object, $geoNear with distance_km, role checks, referential checks.
3	Exercises	**3. Exercises**: Filter exercises · Search exercises (text index) · Top rated exercises · Create exercise (trainer) · Create exercise - duplicate title (409)	$in filters, full-text index, aggregation for the ratings.
4	Reviews	**4. Exercise reviews**: Add review (201) · Add review - already reviewed (409) · Update review - not the author (403) · Delete review as admin (moderation)	Embedded sub-documents, one review per member, ownership rules.
5	Members	**5. Members**: List members (admin) · Get another member - forbidden (403) · Update own profile · Renew membership (admin)	Self-or-admin access, BMI recalculated, membership extended from end date.
6	Workouts and stats	**6. Workouts and stats**: Log workout - rule violations (400) · Log workout (201) · Member stats - seeded member	Cross-field rules (max_bpm ≥ avg_bpm, no future dates), the stats aggregation.
7	Classes	**7. Classes**: List upcoming classes · Create class (201) · Create class - trainer clash (409)	Timetable with spaces_left, trainer clash detection.
8	Bookings (main feature)	**8. Bookings**: Book class (spaces_left 0) · Book class - full (409) · Increase class capacity · Book class - second member · Cancel booking · Cancel booking - already cancelled (409) · Mark attendance	Atomic booking: show the update_one filter in bookings.py. Optionally show booked_count changing in Atlas / Compass.
9	Reports	**9. Reports**: Popular classes · Branch usage · Peak times · Trainer performance · Popular classes - trainer forbidden (403)	Admin-only aggregation pipelines.
10	Errors and logout	**10. General errors and clean-up**: Unknown URL (404) · Wrong method (405) · Malformed JSON body (400) · Logout member · Use logged-out token (401)	Every error has the same JSON shape; token blacklist.
11	Full test run	Collection → Run → all folders.	153 requests, 325 assertions, 0 failures.

Before and during recording

- If the data is more than about 3 weeks old, re-run **seed.py** first (see the note in 5.3).
- Do one full run of the collection off camera to make sure everything is green.
- Tokens last only 8 minutes. Before each scene, re-send the Login requests you need (off camera is fine). If you record in several sessions, start each session by re-sending folder 1 requests 1–11 off

camera.

- Scenes 1 to 10 must be recorded in that order, because each depends on variables saved by earlier scenes.
- Zoom Postman (Ctrl + =) so the status code and the JSON can be read in the recording.
- For each request say three things: who is calling (which token), what you expect (status code), and why.

Appendix A. All 153 Postman requests in run order

Send these top to bottom. **Auth** is the token sent in x-access-token (Basic = Basic Auth login, – = none). **Status** is the expected status code. **Saves** lists the collection variables the request's script stores for later requests.

1. Auth

#	Request	Method and URL	Auth	Status	Saves
1	Login as admin	`GET /auth/login`	Basic	200	adminToken, runId
2	Login as trainer	`GET /auth/login`	Basic	200	trainerId, trainerToken
3	Login - wrong password	`GET /auth/login`	Basic	401	
4	Login - no credentials	`GET /auth/login`	–	401	
5	Current user (trainer)	`GET /auth/me`	trainerToken	200	branchId
6	Register - validation errors	`POST /auth/register`	–	400	
7	Register - username and email taken	`POST /auth/register`	–	409	
8	Register new member	`POST /auth/register`	–	201	memberId
9	Login as new member	`GET /auth/login`	Basic	200	memberToken
10	Register second member	`POST /auth/register`	–	201	otherMemberId
11	Login as second member	`GET /auth/login`	Basic	200	otherMemberToken
12	Current user (member)	`GET /auth/me`	memberToken	200	
13	Current user - missing token	`GET /auth/me`	–	401	
14	Current user - invalid token	`GET /auth/me`	not.a.valid-token	401	

2. Branches

#	Request	Method and URL	Auth	Status	Saves
15	List branches	`GET /branches`	–	200	
16	List branches - filter by city and facility	`GET /branches?city=Belfast&facility=Lockers`	–	200	
17	List branches - invalid query	`GET /branches?ps=500&sort=name`	–	400	
18	Branches near a location	`GET /branches/near?lat=54.597&lng=-5.93&km=5`	–	200	
19	Branches near - missing lat	`GET /branches/near?lng=-5.93`	–	400	
20	Branches near - out of range	`GET /branches/near?lat=54.6&lng=-5.93&km=250`	–	400	
21	Get branch	`GET /branches/{{branchId}}`	–	200	
22	Get branch - invalid id	`GET /branches/not-an-id`	–	400	
23	Get branch - not found	`GET /branches/000000000000000000000000`	–	404	
24	Branch upcoming classes	`GET /branches/{{branchId}}/classes?category=Yoga,Strength&ps=5`	–	200	
25	Create branch - no token	`POST /branches`	–	401	
26	Create branch - member forbidden	`POST /branches`	memberToken	403	
27	Create branch - validation errors	`POST /branches`	adminToken	400	
28	Create branch	`POST /branches`	adminToken	201	newBranchId
29	Create branch - duplicate name	`POST /branches`	adminToken	409	
30	Update branch	`PUT /branches/{{newBranchId}}`	adminToken	200	
31	Update branch - empty body	`PUT /branches/{{newBranchId}}`	adminToken	400	
32	Delete branch - still in use	`DELETE /branches/{{branchId}}`	adminToken	409	
33	Delete branch	`DELETE /branches/{{newBranchId}}`	adminToken	204	
34	Get deleted branch	`GET /branches/{{newBranchId}}`	–	404	

3. Exercises

#	Request	Method and URL	Auth	Status	Saves
35	List exercises	`GET /exercises`	–	200	
36	Filter exercises	`GET /exercises?body_part=Chest,Triceps&equipment=Barbell&level=Beginner&min_rating=7&sort=rating&ps=20`	–	200	
37	Search exercises (text index)	`GET /exercises?q=kettlebell swing&ps=5`	–	200	
38	Filter exercises - invalid values	`GET /exercises?body_part=Wings&min_rating=12&sort=popularity`	–	400	
39	Exercise filter values	`GET /exercises/filters`	–	200	
40	Top rated exercises	`GET /exercises/top-rated?body_part=Chest,Shoulders&min_reviews=2&limit=5`	–	200	
41	Create exercise - member forbidden	`POST /exercises`	memberToken	403	
42	Create exercise - validation errors	`POST /exercises`	trainerToken	400	
43	Create exercise	`POST /exercises`	trainerToken	201	exerciseId
44	Create exercise - duplicate title	`POST /exercises`	adminToken	409	
45	Get exercise	`GET /exercises/{{exerciseId}}`	–	200	
46	Update exercise	`PUT /exercises/{{exerciseId}}`	adminToken	200	
47	Update exercise - invalid	`PUT /exercises/{{exerciseId}}`	adminToken	400	

4. Exercise reviews

#	Request	Method and URL	Auth	Status	Saves
48	Add review - staff cannot review	`POST /exercises/{{exerciseId}}/reviews`	adminToken	403	
49	Add review - validation errors	`POST /exercises/{{exerciseId}}/reviews`	memberToken	400	
50	Add review	`POST /exercises/{{exerciseId}}/reviews`	memberToken	201	reviewId
51	Add review - already reviewed	`POST /exercises/{{exerciseId}}/reviews`	memberToken	409	
52	List reviews	`GET /exercises/{{exerciseId}}/reviews?sort=stars`	–	200	
53	Get review	`GET /exercises/{{exerciseId}}/reviews/{{reviewId}}`	–	200	
54	Get review - not found	`GET /exercises/{{exerciseId}}/reviews/000000000000000000000000`	–	404	
55	Update review	`PUT /exercises/{{exerciseId}}/reviews/{{reviewId}}`	memberToken	200	
56	Update review - not the author	`PUT /exercises/{{exerciseId}}/reviews/{{reviewId}}`	otherMemberToken	403	
57	Delete review - not the author	`DELETE /exercises/{{exerciseId}}/reviews/{{reviewId}}`	otherMemberToken	403	
58	Delete own review	`DELETE /exercises/{{exerciseId}}/reviews/{{reviewId}}`	memberToken	204	
59	Get deleted review	`GET /exercises/{{exerciseId}}/reviews/{{reviewId}}`	–	404	
60	Add review again	`POST /exercises/{{exerciseId}}/reviews`	memberToken	201	reviewId
61	Delete review as admin (moderation)	`DELETE /exercises/{{exerciseId}}/reviews/{{reviewId}}`	adminToken	204	

5. Members

#	Request	Method and URL	Auth	Status	Saves
62	List members (admin)	`GET /members?status=active&plan=Monthly,Annual&workout_type=Yoga&ps=5`	adminToken	200	seededMemberId
63	Search members by name or email	`GET /members?q=postman.{{runId}}`	adminToken	200	
64	List members - member forbidden	`GET /members`	memberToken	403	
65	Get own member record	`GET /members/{{memberId}}`	memberToken	200	
66	Get another member - forbidden	`GET /members/{{otherMemberId}}`	memberToken	403	
67	Get member - not found (admin)	`GET /members/000000000000000000000000`	adminToken	404	
68	Update own profile	`PUT /members/{{memberId}}`	memberToken	200	
69	Update own email - forbidden	`PUT /members/{{memberId}}`	memberToken	403	
70	Update profile - validation errors	`PUT /members/{{memberId}}`	memberToken	400	
71	Update email - already used (admin)	`PUT /members/{{memberId}}`	adminToken	409	
72	Renew membership (admin)	`PUT /members/{{memberId}}/membership`	adminToken	200	
73	Renew membership - invalid plan	`PUT /members/{{memberId}}/membership`	adminToken	400	
74	Renew membership - member forbidden	`PUT /members/{{memberId}}/membership`	memberToken	403	
75	Create member (admin)	`POST /members`	adminToken	201	adminMemberId
76	Create member - duplicate email	`POST /members`	adminToken	409	
77	Create member - unknown branch	`POST /members`	adminToken	400	
78	Delete member (admin)	`DELETE /members/{{adminMemberId}}`	adminToken	204	

6. Workouts and stats

#	Request	Method and URL	Auth	Status	Saves
79	Log workout - rule violations	`POST /members/{{memberId}}/workouts`	memberToken	400	workoutDate, workoutFrom
80	Log workout - invalid nested exercise	`POST /members/{{memberId}}/workouts`	memberToken	400	
81	Log workout - unknown exercise	`POST /members/{{memberId}}/workouts`	memberToken	400	
82	Log workout - other member forbidden	`POST /members/{{otherMemberId}}/workouts`	memberToken	403	
83	Log workout	`POST /members/{{memberId}}/workouts`	memberToken	201	workoutId
84	List workouts	`GET /members/{{memberId}}/workouts`	memberToken	200	
85	List workouts - filter by type and date range	`GET /members/{{seededMemberId}}/workouts?type=Yoga,Strength&from={{workoutFrom}}&sort=oldest`	adminToken	200	
86	List workouts - from after to	`GET /members/{{memberId}}/workouts?from=2026-10-01&to=2026-09-01`	memberToken	400	
87	Get workout	`GET /members/{{memberId}}/workouts/{{workoutId}}`	memberToken	200	
88	Update workout	`PUT /members/{{memberId}}/workouts/{{workoutId}}`	memberToken	200	
89	Update workout - breaks bpm rule	`PUT /members/{{memberId}}/workouts/{{workoutId}}`	memberToken	400	
90	Member stats	`GET /members/{{memberId}}/stats?weeks=4`	memberToken	200	
91	Member stats - seeded member	`GET /members/{{seededMemberId}}/stats`	adminToken	200	
92	Member stats - invalid weeks	`GET /members/{{memberId}}/stats?weeks=100`	memberToken	400	
93	Delete workout	`DELETE /members/{{memberId}}/workouts/{{workoutId}}`	memberToken	204	
94	Get deleted workout	`GET /members/{{memberId}}/workouts/{{workoutId}}`	memberToken	404	

7. Classes

#	Request	Method and URL	Auth	Status	Saves
95	List upcoming classes	`GET /classes?ps=5`	–	200	seededClassId
96	List available classes at a branch	`GET /classes?branch={{branchId}}&available=true&category=Yoga,HIIT,Cardio,Strength&ps=20`	–	200	
97	List past classes by level	`GET /classes?when=past&level=Beginner,All levels&ps=5`	–	200	
98	List classes - invalid query	`GET /classes?available=maybe&level=Pro&branch=123`	–	400	
99	Get class	`GET /classes/{{seededClassId}}`	–	200	
100	Get class - not found	`GET /classes/000000000000000000000000`	–	404	
101	Create class - member forbidden	`POST /classes`	memberToken	403	clashStart, classStart
102	Create class - validation errors	`POST /classes`	trainerToken	400	
103	Create class - start time in the past	`POST /classes`	trainerToken	400	
104	Create class	`POST /classes`	trainerToken	201	classId
105	Create class - trainer clash	`POST /classes`	trainerToken	409	
106	Update class	`PUT /classes/{{classId}}`	trainerToken	200	
107	Update class - cannot complete a future class	`PUT /classes/{{classId}}`	trainerToken	400	

8. Bookings

#	Request	Method and URL	Auth	Status	Saves
108	Book class - staff cannot book	`POST /classes/{{classId}}/bookings`	adminToken	403	
109	Book class	`POST /classes/{{classId}}/bookings`	memberToken	201	bookingId
110	Book class - already booked	`POST /classes/{{classId}}/bookings`	memberToken	409	
111	Book class - full	`POST /classes/{{classId}}/bookings`	otherMemberToken	409	
112	Increase class capacity	`PUT /classes/{{classId}}`	trainerToken	200	
113	Book class - second member	`POST /classes/{{classId}}/bookings`	otherMemberToken	201	otherBookingId
114	Reduce capacity below bookings	`PUT /classes/{{classId}}`	trainerToken	409	
115	List class bookings (trainer)	`GET /classes/{{classId}}/bookings`	trainerToken	200	
116	List class bookings - member forbidden	`GET /classes/{{classId}}/bookings`	memberToken	403	
117	Get own booking	`GET /classes/{{classId}}/bookings/{{bookingId}}`	memberToken	200	
118	Get another member's booking - forbidden	`GET /classes/{{classId}}/bookings/{{bookingId}}`	otherMemberToken	403	
119	Mark attendance - class not started	`PUT /classes/{{classId}}/bookings/{{bookingId}}/attendance`	trainerToken	400	
120	Cancel booking - not yours	`DELETE /classes/{{classId}}/bookings/{{bookingId}}`	otherMemberToken	403	
121	Cancel booking	`DELETE /classes/{{classId}}/bookings/{{bookingId}}`	memberToken	200	
122	Cancel booking - already cancelled	`DELETE /classes/{{classId}}/bookings/{{bookingId}}`	memberToken	409	
123	Member's bookings	`GET /members/{{memberId}}/bookings?when=upcoming`	memberToken	200	
124	Member's bookings - forbidden	`GET /members/{{otherMemberId}}/bookings`	memberToken	403	
125	Delete class - has active bookings	`DELETE /classes/{{classId}}`	trainerToken	409	
126	Cancel class	`PUT /classes/{{classId}}`	trainerToken	200	
127	Book cancelled class	`POST /classes/{{classId}}/bookings`	memberToken	400	
128	Delete class	`DELETE /classes/{{classId}}`	trainerToken	204	
129	Find a past class	`GET /classes?when=past&ps=1`	–	200	pastClassId
130	List past class bookings (admin)	`GET /classes/{{pastClassId}}/bookings?status=booked`	adminToken	200	pastBookingId
131	Mark attendance	`PUT /classes/{{pastClassId}}/bookings/{{pastBookingId}}/attendance`	adminToken	200	
132	Mark attendance - invalid body	`PUT /classes/{{pastClassId}}/bookings/{{pastBookingId}}/attendance`	adminToken	400	
133	Book past class	`POST /classes/{{pastClassId}}/bookings`	memberToken	400	

9. Reports

#	Request	Method and URL	Auth	Status	Saves
134	Popular classes	`GET /reports/popular-classes?limit=5`	adminToken	200	
135	Popular classes - filtered by category and sorted by attendance	`GET /reports/popular-classes?category=Yoga,HIIT&sort=attendance_rate&to={{workoutDate}}`	adminToken	200	
136	Popular classes - invalid query	`GET /reports/popular-classes?sort=profit&from=2026-10-10&to=2026-01-01`	adminToken	400	
137	Popular classes - trainer forbidden	`GET /reports/popular-classes`	trainerToken	403	
138	Branch usage	`GET /reports/branch-usage`	adminToken	200	
139	Peak times	`GET /reports/peak-times?limit=5`	adminToken	200	
140	Trainer performance	`GET /reports/trainer-performance?sort=fill_rate`	adminToken	200	
141	Membership summary	`GET /reports/membership-summary?days=30&limit=5`	adminToken	200	
142	Membership summary - invalid days	`GET /reports/membership-summary?days=400`	adminToken	400	

10. General errors and clean-up

#	Request	Method and URL	Auth	Status	Saves
143	Unknown URL	`GET /does-not-exist`	–	404	
144	Wrong method	`DELETE /reports/peak-times`	adminToken	405	
145	Malformed JSON body	`PUT /members/{{memberId}}`	memberToken	400	
146	Delete exercise	`DELETE /exercises/{{exerciseId}}`	trainerToken	204	
147	Get deleted exercise	`GET /exercises/{{exerciseId}}`	–	404	
148	Logout member	`GET /auth/logout`	memberToken	200	
149	Use logged-out token	`GET /auth/me`	memberToken	401	
150	Delete registered member (admin)	`DELETE /members/{{memberId}}`	adminToken	204	
151	Delete second member (admin)	`DELETE /members/{{otherMemberId}}`	adminToken	204	
152	Login as deleted member	`GET /auth/login`	Basic	401	
153	Logout admin	`GET /auth/logout`	adminToken	200	